# Neuro-Fuzzy Computing - Problem-set 1st: Teamwork

GEORGIOS MOIROTSOS - 03401
GEORGIOS MASTELLOS - 03402

November 04 - December 01, 2024

## Problem 01

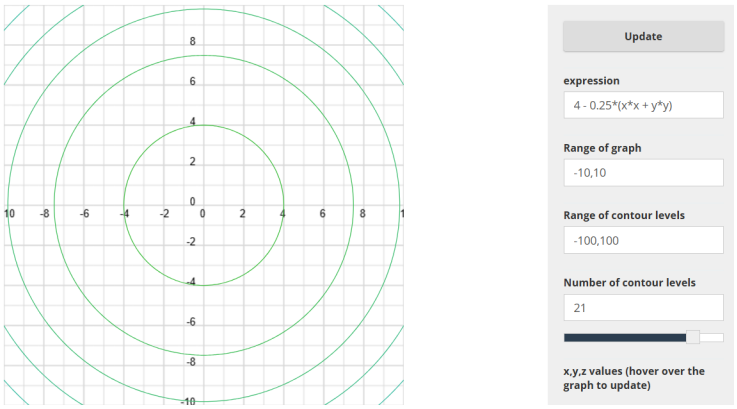



Figure 1: contour lines

f(x,y) $= 4 - 0.25(x^2 + y^2)$

quadratic form: f(x,y) $= ax^2 + by^2 + cxy + dx + ey + f$

f(x,y) $= 4 - 0.25(x^2 + y^2) -> f(x, y) = -0.25x^2 - 0.25y^2 + 4$

Find the local minima/maxima of f(x,y):

Methodology:

1. $fx = 0, fy = 0$
2. $D = fxx * fyy - f^2xy$

3. i) $D > 0,\ fxx > 0,\ f(a,b) -> localmin$
   ii)$D > 0,\ fxx < 0,\ f(a,b) -> localmax$
   iii)$D < 0,\ f(a,b) -> saddle - point$

Solution:

1. $fx = -0.5 * x = 0 -> x = 0$
   $fy = -0.5 * y = 0 -> y = 0$
2. $fxx = -0.5, fyy = -0.5, fxy = 0$
   $D = -0.5 * (-0.5) = 0.25$
3. $D = 0.25 > 0, fxx = -0.5 < 0$
   So, f(0,0) = 4 is a local max

# Problem 02

The given function is:

$$f(x_1, x_2) = 0.26(x_1^2 + x_2^2) - 0.48(x_1 x_2).$$

## Step 1: Gradient of $f(x_1, x_2)$

The gradient of $f(x_1, x_2)$ is:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 0.52x_1 - 0.48x_2 \\ 0.52x_2 - 0.48x_1 \end{bmatrix}.$$

At the initial point $x_0 = (2, 3)$:

$$\nabla f(x_0) = \begin{bmatrix} 0.52(2) - 0.48(3) \\ 0.52(3) - 0.48(2) \end{bmatrix} = \begin{bmatrix} 1.04 - 1.44 \\ 1.56 - 0.96 \end{bmatrix} = \begin{bmatrix} -0.4 \\ 0.6 \end{bmatrix}.$$

## Step 2: Compute the Direction

The magnitude of $\nabla f(x_0)$ is:

$$\|\nabla f(x_0)\| = \sqrt{\nabla f(x_0)^\top \nabla f(x_0)} = \sqrt{(-0.4)^2 + (0.6)^2} = \sqrt{0.16 + 0.36} = \sqrt{0.52}.$$

The direction is:

$$S_0 = -\frac{\nabla f(x_0)}{\|\nabla f(x_0)\|} = -\frac{1}{\sqrt{0.52}} \begin{bmatrix} -0.4 \\ 0.6 \end{bmatrix} = \begin{bmatrix} 0.4/\sqrt{0.52} \\ -0.6/\sqrt{0.52} \end{bmatrix}.$$

## Step 3: Update Formula

Using the update formula:

$$x_1 = x_0 + \lambda_0 S_0,$$

we have:

$$x_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + \lambda_0 \begin{bmatrix} 0.4/\sqrt{0.52} \\ -0.6/\sqrt{0.52} \end{bmatrix}.$$

### Step 4: Minimizing $f(x_1)$

The function $f(x_1)$ becomes:

$$f(x_1) = 0.26\left( \left(2 + \lambda_0 \frac{0.4}{\sqrt{0.52}}\right)^2 + \left(3 - \lambda_0 \frac{0.6}{\sqrt{0.52}}\right)^2 \right) - 0.48\left( \left(2 + \lambda_0 \frac{0.4}{\sqrt{0.52}}\right)\left(3 - \lambda_0 \frac{0.6}{\sqrt{0.52}}\right) \right).$$

Taking the derivative with respect to $\lambda_0$, we find:

$$\frac{df(x_1)}{d\lambda_0} = \frac{313}{325}\lambda_0 - \frac{\sqrt{13}}{5}.$$

Setting $\frac{df(x_1)}{d\lambda_0} = 0$, we solve for $\lambda_0$:

$$\lambda_0 = \frac{325\sqrt{13}}{313 \cdot 5} = \frac{65\sqrt{13}}{313}.$$

### Step 5: Update $x_1$

Substitute $\lambda_0$ back into the update formula:

$$x_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix} - 0.745 \begin{bmatrix} -0.4/\sqrt{0.52} \\ 0.6/\sqrt{0.52} \end{bmatrix}.$$

Simplify:

$$x_1 = \begin{bmatrix} 2 + 0.3 \\ 3 - 0.45 \end{bmatrix} = \begin{bmatrix} 2.41 \\ 2.37 \end{bmatrix}.$$

## Iteration 2: Starting at $\mathbf{x}_1 = (2.41, 2.37)$

### Step 1: Compute the Gradient

The gradient of $f(x_1, x_2)$ is:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 0.52x_1 - 0.48x_2 \\ 0.52x_2 - 0.48x_1 \end{bmatrix}.$$

At $\mathbf{x}_1 = (2.41, 2.37)$:

$$\nabla f(2.41, 2.37) = \begin{bmatrix} 0.52(2.41) - 0.48(2.37) \\ 0.52(2.37) - 0.48(2.41) \end{bmatrix} = \begin{bmatrix} 1.2532 - 1.1376 \\ 1.2324 - 1.1568 \end{bmatrix} = \begin{bmatrix} 0.12 \\ 0.08 \end{bmatrix}.$$

### Step 2: Compute the Direction

The magnitude of the gradient is:

$$\|\nabla f(x_1)\| = \sqrt{(0.12)^2 + (0.08)^2} = \sqrt{0.0144 + 0.0064} = \sqrt{0.02} = 0.141.$$

The direction is:

$$S_1 = -\frac{\nabla f(x_1)}{\|\nabla f(x_1)\|} = -\frac{1}{0.141} \begin{bmatrix} 0.12 \\ 0.08 \end{bmatrix} = \begin{bmatrix} -0.85 \\ -0.57 \end{bmatrix}.$$

### Step 3: Update Formula

The update formula is:

$$x_2 = x_1 + \lambda_1 S_1.$$

Substituting the values:

$$x_2 = \begin{bmatrix} 2.41 \\ 2.37 \end{bmatrix} - \lambda_1 \begin{bmatrix} 0.12/0.144 \\ 0.08/0.144 \end{bmatrix}.$$

### Step 4: Minimizing $f(x_2)$

The function $f(x_2)$ becomes:

$$f(x_2) = 0.26\left( \left(2.41 - \lambda_1 \frac{0.12}{0.144}\right)^2 + \left(2.37 - \lambda_1 \frac{0.08}{0.144}\right)^2 \right) - 0.48\left( \left(2.41 - \lambda_1 \frac{0.12}{0.144}\right)\left(2.37 - \lambda_1 \frac{0.08}{0.144}\right) \right).$$

Taking the derivative:

$$\frac{df(x_2)}{d\lambda_1} = \frac{25}{324}\lambda_1 - \frac{83}{600}.$$

Setting $\frac{df(x_2)}{d\lambda_1} = 0$, solve for $\lambda_1$:

$$\lambda_1 = \frac{83 \cdot 324}{600 \cdot 25} = 1.79.$$

### Step 5: Update $x_2$

Substitute $\lambda_1 = 1.79$ into the update formula:

$$x_2 = \begin{bmatrix} 2.41 \\ 2.37 \end{bmatrix} - 1.79 \begin{bmatrix} 0.12/0.144 \\ 0.08/0.144 \end{bmatrix}.$$

Simplify:

$$x_2 = \begin{bmatrix} 2.41 - 1.79(0.12/0.144) \\ 2.37 - 1.79(0.08/0.144) \end{bmatrix} = \begin{bmatrix} 0.92 \\ 1.37 \end{bmatrix}.$$

### Final Result

After two iterations, the updated point is:

$$x_2 = \begin{bmatrix} 0.92 \\ 1.37 \end{bmatrix}.$$

# Problem 03

The code implementetion of Problem 03 exists in the file problem03.ipynb. For better images, please run the code.

$x_{k+1} = f(x_k) = c * x_k * (1 - x_k)$

x values in (0,1) and $0 < c <= 4$

**A.**

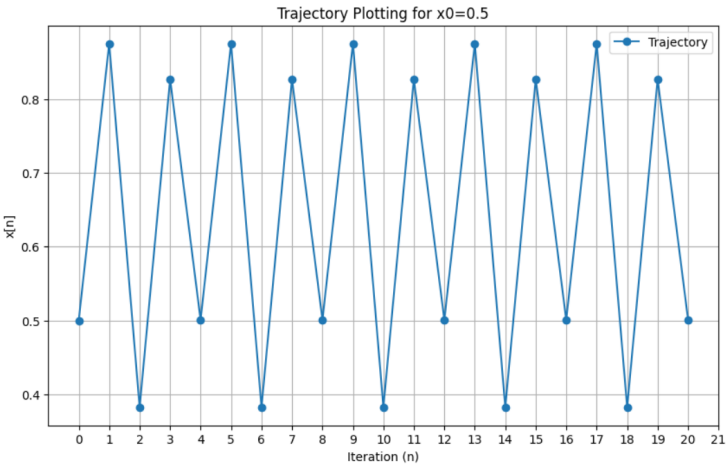



Figure 2: $x_0 = 0.5$

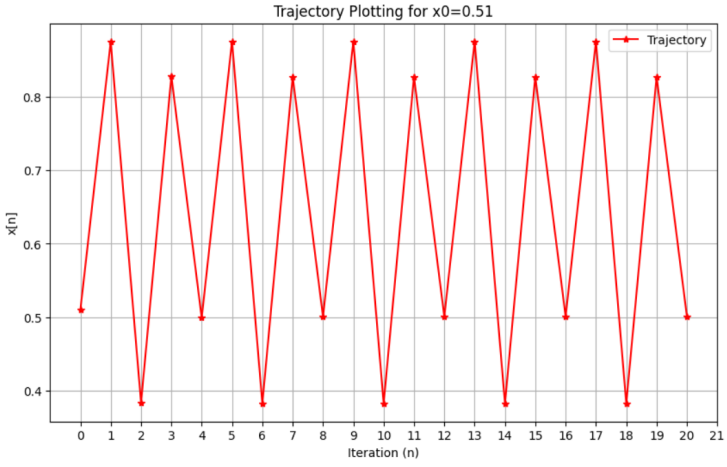



Figure 3: $x_0 = 0.51$

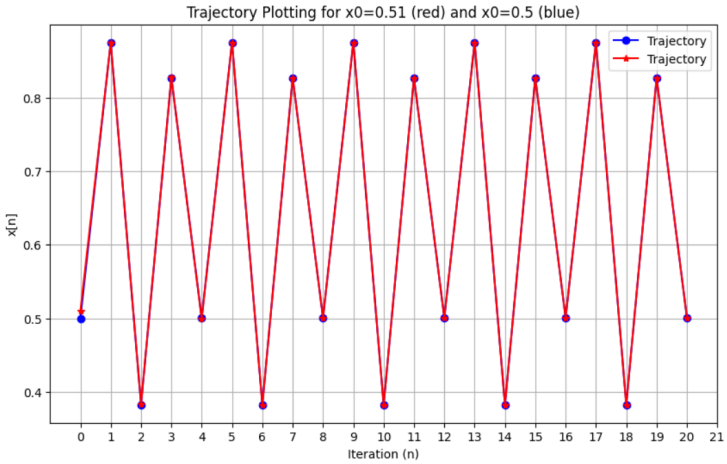



Figure 4: $x_0 = 0.8$ in blue, $x_0 = 0.51$ in red

Watching the first two plots ($x_0 = 0.5$ in blue, $x_0 = 0.51$ in red) we end up to the following conclusions:

1. The two trajectories are periodic. That's because the values of x[n] are pretty much repeated. Also, the behavior of our system is repeated.
2. A difference in 0.01 for $x_0 = 0.5$ and $x_0 = 0.51$ doesn't affect the overal shape of the discrete trajectory. The red plot has overlap the blue one, which means that the two plots are pretty much similar.

**B.** Now, we will run the same code for c=4

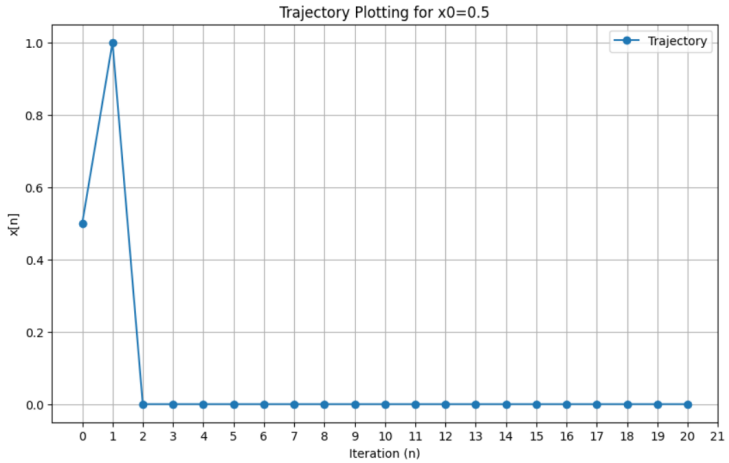



Figure 5: $x_0 = 0.5$

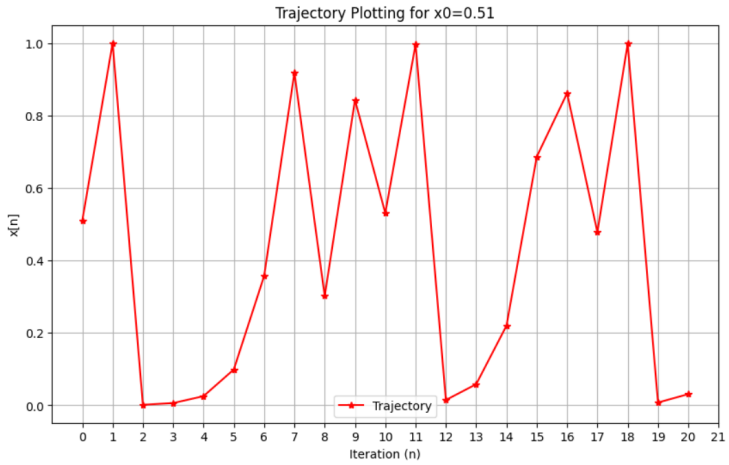



Figure 6: $x_0 = 0.51$

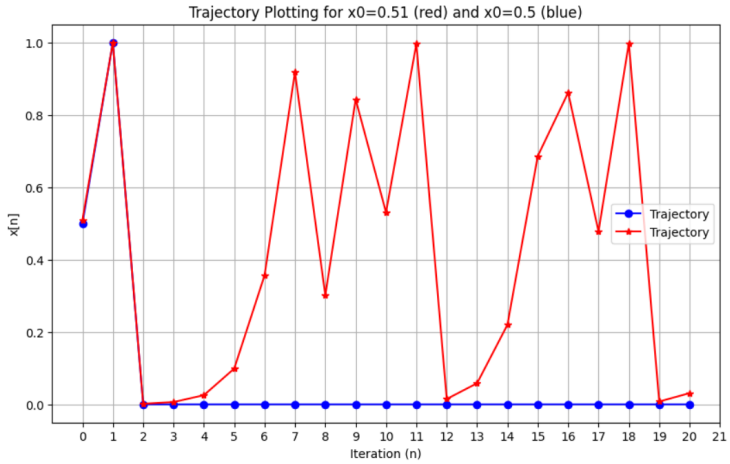
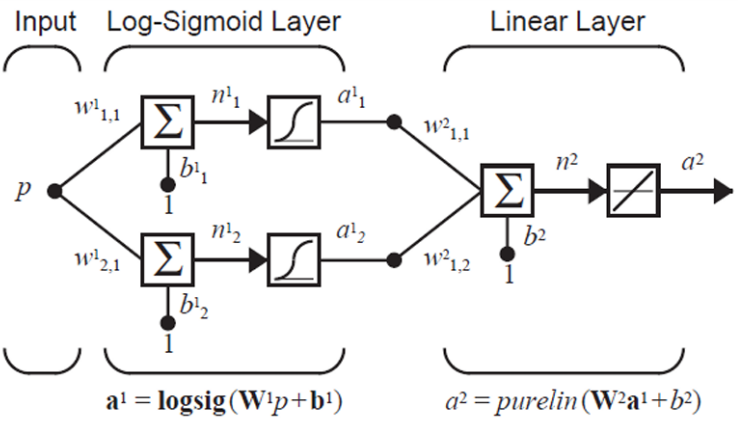



Figure 7: $x_0 = 0.5$ in blue, $x_0 = 0.51$ in red

As we can see, **both trajectories are aperiodic**.Values do not repeat themselves (even in a predictable pattern) and the sequence shows a chaotic behavior.To sum up, **c=4 is definitely produce chaos.**

# Problem 04

Consider the following neural network




**with**

$$w^1_{1,1} = -3, w^1_{2,1} = -1.5, b^1_1 = -1, b^1_2 = -0.25, w^2_{1,1} = 1, w^2_{1,2} = 1.5, b^2 = 2$$

**Sketch the following responses (plot the indicated variable versus p for -3 ¡ p ¡ 3).**

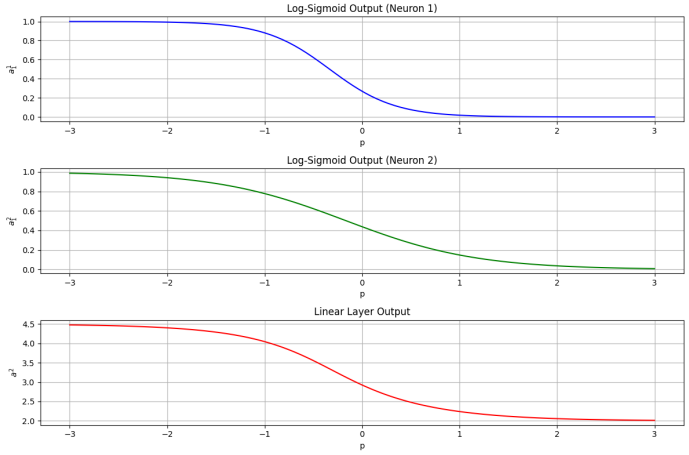
**A**. $a_1^1$

**B**. $a_2^1$

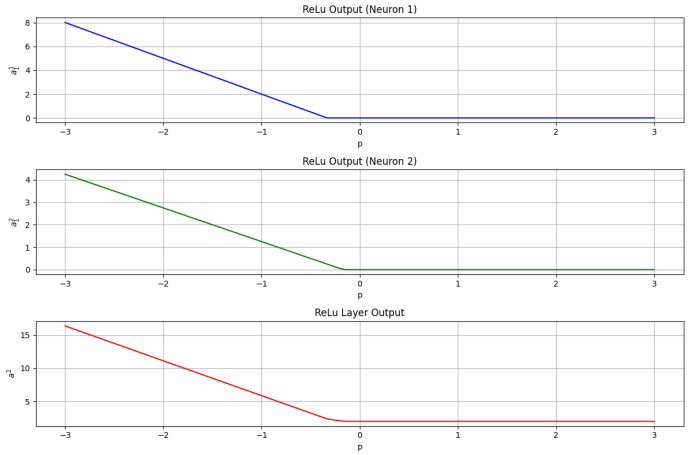
**C**. $a^2$

Then, change the logsig activation function with the ReLU activation function and sketch again the aforementioned responses.

## Step 1: Results for logsig




## Step 2: Results for ReLu



# Problem 05

$E = \frac{1}{2} * \sum(t_k - y_k)^2 + \sum(w_{j,i})^2$

derive the gradient descent rule

- $\frac{1}{2} * \sum(t_k - y_k)^2$ : squared error
- $t_k$ : target
- $y_k$ : output
- $\sum(w_{j,i})^2$ : weight regularization term
- **Gradient Descent** : mathematical technique used for approximating integrals and optimizing functions. Also, used for finding the minimum of a function by following the direction of the negative gradient.

- We need to minimize E with respect to the weights $w_{j,i}$
$\frac{\partial E}{\partial w_{j,i}} = \frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial w_{j,i}} + \sum \frac{\partial (w_{j,i})^2}{\partial w_{j,i}}$

- $\frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial w_{j,i}} = \frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial y_k} * \frac{\partial y_k}{\partial w_{j,i}}$
- $\frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial y_k} = -\sum(t_k - y_k)$
- $\frac{\partial y_k}{\partial w_{j,i}} = \frac{\partial y_k}{\partial n_j} * \frac{\partial n_j}{\partial w_{j,i}}$ , where $n_j = \sum(w_{ji} * a_i) + bi$
  So, $\frac{\partial n_j}{\partial w_{j,i}} = a_j$,which is the output of the previous neuron

**From backpropagation**: $y_{k+1} = f(\sum w_{k+1} * a_k + b_{k+1})$
So, $\frac{\partial y_k}{\partial n_j} = f'(n_k)$

To sum up:
$\frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial w_{j,i}} = \frac{1}{2} * \sum \frac{\partial (t_k - y_k)^2}{\partial y_k} * \frac{\partial y_k}{\partial n_j} * \frac{\partial n_j}{\partial w_{j,i}} = -\sum(t_k - y_k) * f'(n_k) * a_j$ (1)
($f'(n_k)$ depends on the activation function)

- $\sum \frac{\partial (w_{j,i})^2}{\partial w_{j,i}} = 2 * w_{j,i}$ (2)

From (1),(2): $\frac{\partial E}{\partial w_{j,i}} = -\sum(t_k - y_k) * f'(n_k) * a_j + 2 * w_{j,i}$

From **lecture 3**, the gradient descent can be written as :

$w_{k+1} = w_k - c * \frac{\partial J(w,x)}{\partial w}$ , where $\frac{\partial J(w,x)}{\partial w} = \frac{\partial E}{\partial w_{j,i}}$

The final result is:
$w_{ji} = w_{ji} - c * \frac{\partial E}{\partial w_{j,i}} = w_{ji} - c * [-\sum(t_k - y_k) * f'(n_k) * a_j + 2 * w_{j,i}]$

# Problem 06



**Problem-06**

You are given the following activation function:

$$f(x) = \ln(1 + e^{A(x+B)+Cx^2}) - \ln(1 + e^{D(x-B)}) + E$$

where, A,B,C,D and E are trainable parameters.

A. Show (graphically and analytically) how this activation function can approximate, ReLU and sigmoid.

## Optimization Results

The optimization process finds the following optimal parameters:

$$A = 1.2224, \quad B = -0.3258, \quad C = -0.0144, \quad D = 0.0561, \quad E = 0.5272.$$

## Visualization

The graph below shows the approximated function $f(x)$ (dashed line) and the actual ReLU function (solid line), using MSE. The two functions align closely, particularly in the positive domain, demonstrating the effectiveness of the approximation.

## Conclusion

The optimization process successfully adjusted the parameters of $f(x)$ to approximate the ReLU function with high accuracy. The resulting function $f(x)$ closely follows the ReLU, particularly in the positive domain, while maintaining a smooth transition around zero.

## Likewise for sigmoid function

$$A = 1.0174, \quad B = 0.4912, \quad C = 0.000, \quad D = 1.0175, \quad E = 0.0002.$$

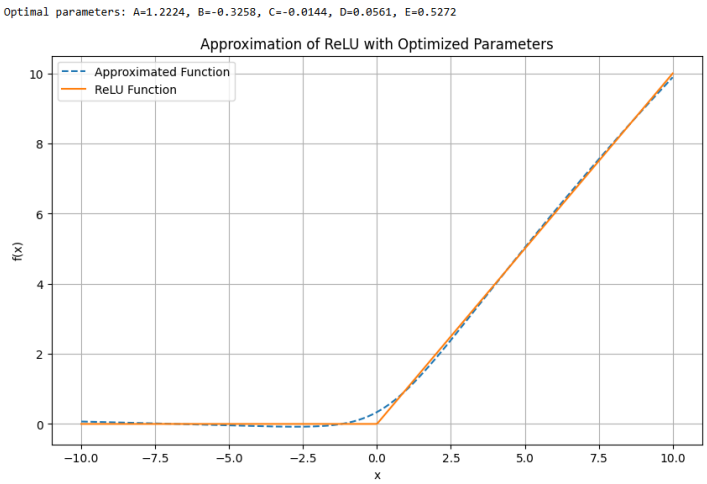



Figure 8: Approximation of ReLU with Optimized Parameters

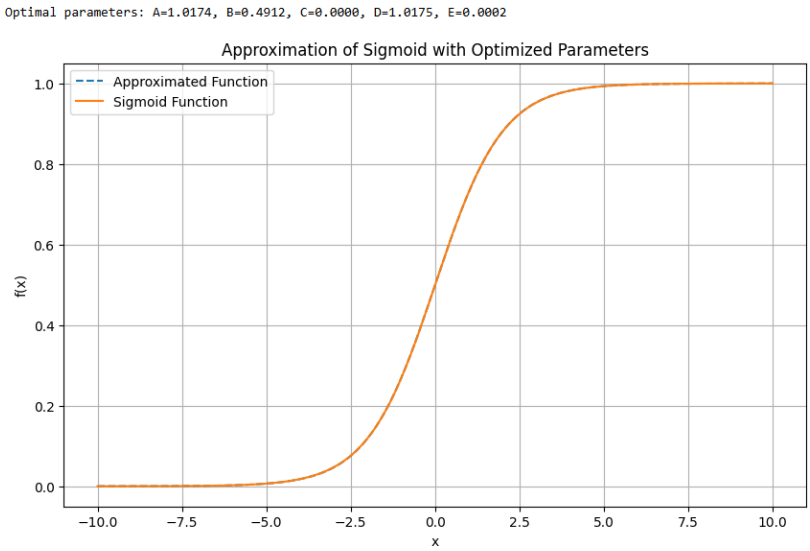



Figure 9: Approximation of sigmoid with Optimized Parameters

# Problem 07



**Problem-07**

Suppose that we have the following four reference patterns and their targets:

$$\left\{\mathbf{p}_1 = \begin{bmatrix}-1\\2\end{bmatrix}, t_1 = -1\right\}\left\{\mathbf{p}_2 = \begin{bmatrix}2\\-1\end{bmatrix}, t_2 = -1\right\}\left\{\mathbf{p}_3 = \begin{bmatrix}0\\-1\end{bmatrix}, t_3 = 1\right\}\left\{\mathbf{p}_4 = \begin{bmatrix}-1\\0\end{bmatrix}, t_4 = 1\right\}$$

The probability of vector $\mathbf{p}_1$ is 0.25, the probability of vector $\mathbf{p}_2$ is 0.35, and the probability of vectors $\mathbf{p}_3$ and $\mathbf{p}_4$ is 0.20.

A. Draw the network diagram for a single-neuron ADALINE network with no bias that could be trained on these patterns.
B. Sketch the contour plot of the mean square error performance index.
C. Show the optimal decision boundary (for the weights that minimize mean square error), and verify (?) that it separates the patterns into the appropriate categories.
D. Find the maximum stable learning rate for the LMS algorithm.

E. Perform one iteration of the LMS algorithm, starting with all weights equal to zero, and presenting input vector $\mathbf{p}_1$. Use a learning rate of $\alpha$=0.005.

Figure 10: Data

## B) Mean Square Error (MSE)

$$F(x) = E(e^2) = c - 2x^\top h + x^\top Q x$$

where:

$$C = E(t^2), \quad h = E(tz), \quad R = E(zz^\top).$$

**Step 1: Calculate $C$**

$$C = E(t^2) = 0.25(t_1)^2 + 0.35(t_2)^2 + 0.20(t_3)^2 + 0.20(t_4)^2.$$

$$c = 0.25 + 0.35 + 0.2 + 0.2 \implies \boxed{c = 1}.$$

**Step 2: Calculate $h$**

$$h = E(tz) = 0.25t_1p_1 + 0.35t_2p_2 + 0.20t_3p_3 + 0.20t_4p_4.$$

Expanding the terms:

$$h = -0.25\begin{bmatrix}-1\\2\end{bmatrix} - 0.35\begin{bmatrix}2\\-1\end{bmatrix} + 0.2\begin{bmatrix}0\\-1\end{bmatrix} + 0.2\begin{bmatrix}-1\\0\end{bmatrix}.$$

Simplifying:

$$h = \begin{bmatrix}0.25 - 0.7 + 0 - 0.2\\-0.5 + 0.35 - 0.2 + 0\end{bmatrix} = \begin{bmatrix}-0.65\\-0.35\end{bmatrix}.$$

Thus:

$$\boxed{h = \begin{bmatrix} -0.65 \\ -0.35 \end{bmatrix}}.$$

**Step 3: Calculate $R$**

$$R = E(zz^\top) = 0.25p_1p_1^\top + 0.35p_2p_2^\top + 0.2p_3p_3^\top + 0.2p_4p_4^\top.$$

Substituting each $p_i$:

$$R = 0.25 \begin{bmatrix} -1 \\ 2 \end{bmatrix} \begin{bmatrix} -1 & 2 \end{bmatrix} + 0.35 \begin{bmatrix} 2 \\ -1 \end{bmatrix} \begin{bmatrix} 2 & -1 \end{bmatrix} + 0.2 \begin{bmatrix} 0 \\ -1 \end{bmatrix} \begin{bmatrix} 0 & -1 \end{bmatrix} + 0.2 \begin{bmatrix} -1 \\ 0 \end{bmatrix} \begin{bmatrix} -1 & 0 \end{bmatrix}.$$

Expanding:

$$R = 0.25 \begin{bmatrix} 1 & -2 \\ -2 & 4 \end{bmatrix} + 0.35 \begin{bmatrix} 4 & -2 \\ -2 & 1 \end{bmatrix} + 0.2 \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} + 0.2 \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

Adding the matrices:

$$R = \begin{bmatrix} 0.25 & -0.5 \\ -0.5 & 1 \end{bmatrix} + \begin{bmatrix} 1.4 & -0.7 \\ -0.7 & 0.35 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0.2 \end{bmatrix} + \begin{bmatrix} 0.2 & 0 \\ 0 & 0 \end{bmatrix}.$$

$$R = \begin{bmatrix} 1.85 & -1.2 \\ -1.2 & 1.55 \end{bmatrix}.$$

Thus:

$$\boxed{R = \begin{bmatrix} 1.85 & -1.2 \\ -1.2 & 1.55 \end{bmatrix}}.$$

**Step 4: Compute $F(x)$**

The function is:

$$F(x) = c - 2x^\top h + x^\top Rx.$$

Substituting:

$$F(x) = 1 - 2 \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} -0.65 \\ -0.35 \end{bmatrix} + \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} 1.85 & -1.2 \\ -1.2 & 1.55 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}.$$

Simplifying:

$$F(x) = 1.85w_1^2 + 1.55w_2^2 - 2.4w_1w_2 + 1.3w_1 + 0.7w_2 + 1.$$

## C)

To minimize $F(x)$, solve:

$$\frac{\partial F}{\partial w_1} = 0 \quad \text{and} \quad \frac{\partial F}{\partial w_2} = 0.$$

Equations:

$$3.7w_1 - 2.4w_2 + 1.3 = 0,$$
$$3.1w_2 - 2.4w_1 + 0.7 = 0.$$

Solving yields:

$$w_1 = -1, \quad w_2 = -1.$$

**Pattern Presentation**

For weights $w_1 = -1$ and $w_2 = -1$:

- Present $p_1$: $a(1) = \text{purelin}(w^\top p_1) = \text{purelin}(\begin{bmatrix}-1 & -1\end{bmatrix}\begin{bmatrix}-1\\2\end{bmatrix}) = -1$,
  $p(1) = t_1 - a(1) = -1 - (-1) = 0$ (Correct).
- Present $p_2$: $a(2) = \begin{bmatrix}-1 & -1\end{bmatrix}\begin{bmatrix}2\\-1\end{bmatrix} = -2 + 1 = -1$,
  $r(2) = t_2 - a(2) = -1 - (-1) = 0$ (Correct).
- Present $p_3$: $a(3) = \begin{bmatrix}-1 & -1\end{bmatrix}\begin{bmatrix}0\\-1\end{bmatrix} = 1$,
  $p(3) = t_3 - a(3) = 1 - 1 = 0$ (Correct).
- Present $p_4$: $a(4) = \begin{bmatrix}-1 & -1\end{bmatrix}\begin{bmatrix}-1\\0\end{bmatrix} = 1$,
  $p(4) = t_4 - a(4) = 1 - 1 = 0$ (Correct).

All patterns are correctly classified.
As long as i have found $w = \begin{bmatrix}-1 & -1\end{bmatrix}$ from minimization i don't do

$$W^{\text{new}} = W^{\text{old}} + 2\alpha e p^\top !!!!$$

## D)

The matrix $A$ is:

$$A = 2 \cdot R = \begin{bmatrix} 3.7 & -2.4 \\ -2.4 & 3.1 \end{bmatrix}.$$

The determinant of $A - 2\lambda I$ is:

$$\det(A - 2\lambda I) = \begin{vmatrix} 3.7 - 2\lambda & -2.4 \\ -2.4 & 3.1 - 2\lambda \end{vmatrix}.$$

Expanding:

$$\det(A - 2\lambda I) = (3.7 - 2\lambda)(3.1 - 2\lambda) - (-2.4)(-2.4).$$

Combine terms:

$$\det(A - 2\lambda I) = 11.47 - 6.8\lambda + 4\lambda^2 - 5.76.$$

Simplify further:

$$4\lambda^2 - 6.8\lambda + 5.71 = 0.$$

The discriminant ($\Delta$) is:

$$\Delta = (-6.8)^2 - 4 \cdot (5.71) = 46.24 - 22.84 = 23.4.$$

The eigenvalues ($\lambda_1, \lambda_2$) are:

$$\lambda_{1,2} = \frac{6.8 \pm \sqrt{23.4}}{8}.$$

Thus:

$$\lambda_1 = 5.818 \quad \text{and} \quad \lambda_2 = 0.881.$$

The maximum scalable learning rate is:

$$\alpha_{\max} = \frac{2}{\lambda_{\max}} = \frac{2}{5.818} \approx 0.3437.$$

## E)

The weight update rule for ADALINE is:

$$W^{\text{new}} = W^{\text{old}} + 2\alpha e p^\top.$$

Given:

$$W(0) = \begin{bmatrix} 0 & 0 \end{bmatrix}, \quad \alpha = 0.005.$$

Presenting $p_1$:

$$a(0) = \text{purelin}(W \cdot p_1) = \text{purelin}((0,0) \cdot [-1,2]^\top) = 0.$$

$$p(0) = t(1) - a(0) = -1 - 0 = -1.$$

Update:

$$W(1) = W(0) + 2\alpha p_1^\top.$$

$$W(1) = \begin{bmatrix} 0 & 0 \end{bmatrix} + 2 \cdot 0.005 \cdot \cdot(-1).$$

$$W(1) = \begin{bmatrix} -0.01 & 0.02 \end{bmatrix}.$$

Thus:

$$\boxed{W(1) = \begin{bmatrix} -0.01 & 0.02 \end{bmatrix}}.$$

# Problem 08

Find a single-layer network that has the same input/output characteristic as the network below.

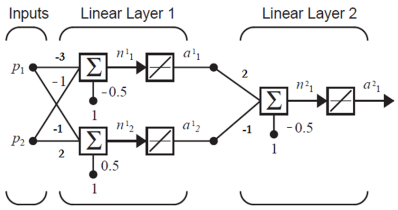



Figure 11: Data

## Step 1: Calculate $n^1_1$ and $n^1_2$

$$n^1_1 = w^1_{1,1} \cdot p_1 + w^1_{2,1} \cdot p_2 + b_1$$

$$n^1_2 = w^1_{1,2} \cdot p_1 + w^1_{2,2} \cdot p_2 + b_2$$

So:

$$n^1_1 = -3 \cdot p_1 - p_2 - 0.5$$

$$n^1_2 = -p_1 + 2 \cdot p_2 + 0.5$$

## Step 2: Calculate $a^1_1$ and $a^1_2$

**Since we have a linear function:**

$$a^1_1 = n^1_1$$

$$a^1_2 = n^1_2$$

Thus:

$$a^1_1 = -3 \cdot p_1 - p_2 - 0.5$$

$$a^1_2 = -p_1 + 2 \cdot p_2 + 0.5$$

So the input of Linear Layer 2 is:

$$n^2_1 = 2 \cdot a^1_1 + (-1) \cdot a^1_2$$

$$n^2_1 = -6 \cdot p_1 - 2 \cdot p_2 - 1 + p_1 - 2 \cdot p_2 - 0.5$$

$$n^2_1 = -5 \cdot p_1 - 4 \cdot p_2 - 2$$

## Step 3: Conclusion

**Since we have a linear function:**

$$a_1^2 = n_1^2 = -5 \cdot p_1 - 4 \cdot p_2 - 2$$

And we can assume that we could have a new neuron one layer with

$w_1^1$ = -5

$w_2^1$ = -4

and $b_1$ = -2

# Problem 09

## A.

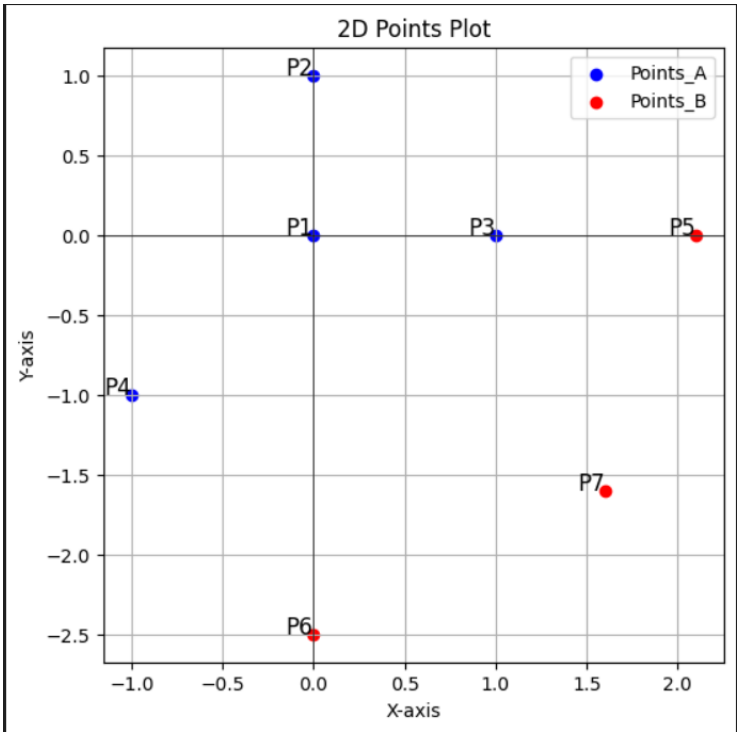



Figure 12: patterns in a 2d diagram

As long as the blue points (class A) are (typically) on the left side, and the red points (class B) on the right side, we can say that the categories are linearly

separable.

## B.

For the aforementioned patterns design the architecture of an ADALINE neural network which will separate the classes.

In order to implement question B, we are going to guess the target values. More specifically, $t_1 = 1$ (class A) and $t_2 = -1$ (class B).

Firstly, we will find **F,c,h,R and x***.

```
h = [[-0.74]
 [ 0.82]]
c = -0.1
R = [[ 1.594 -0.412]
 [-0.412  1.962]]
F(x): w1*(1.594*w1 - 0.412*w2) + 1.48*w1 + w2*(-0.412*w1 + 1.962*w2) - 1.64*w2 - 0.1
x* = [[-0.37665958]
 [ 0.33884621]]
```

Figure 13: F,c,h,R and x*

As long as we have found x*, **we will use these two values for w1 and w2**. Then, we will find outputs "a" (a=w.T * p) and compute the errors **e=t-a**. **By finding these errors, we can understand if the classes are well separated.**

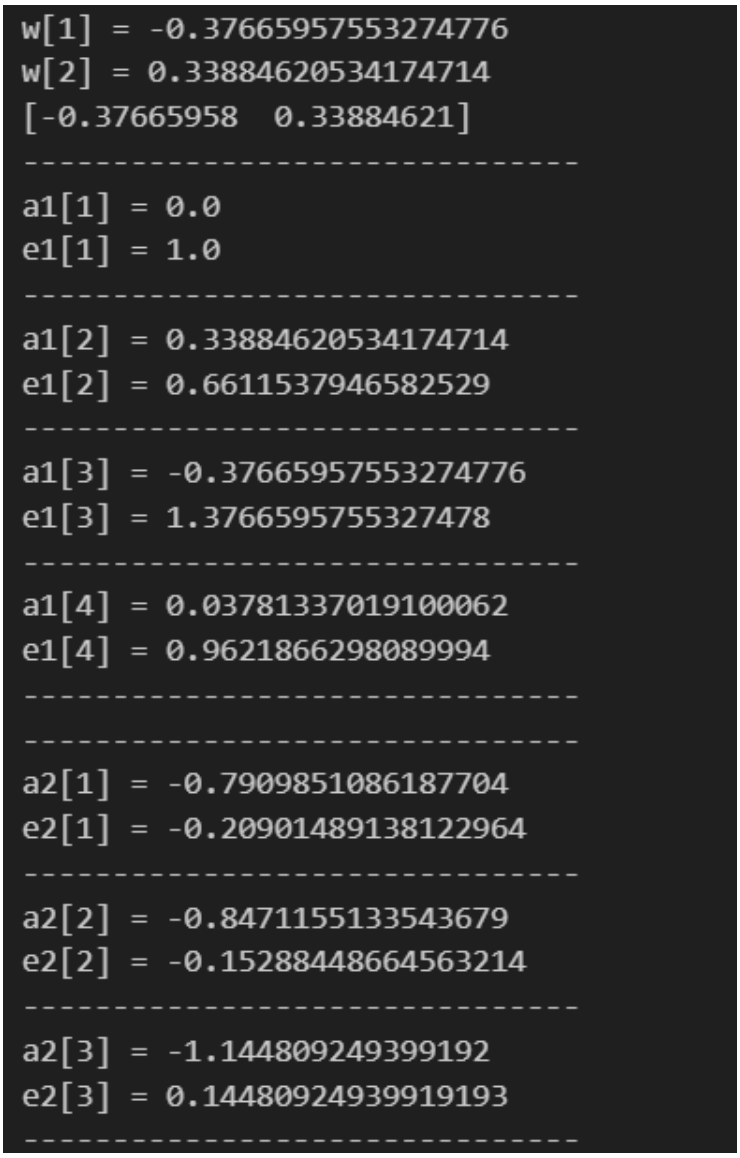



Figure 14: $e_1$ stands for $t_1$,$e_2$ stands for $t_2$ (same for outputs a)

- The closer the value of error "e" is to 0, the better the classes are separated well. Large "e" values indicate poor separation for that point

- Conclusions for each class:
  - **Errors for class A** ($e_1$) are larger than zero. This means that class A for $t_1$ is not well separated. We can change the value of $t_1$ in the beggining of the code, check the errors again and see if the values of $e_1$ are closer to

0.
- **The errors for class B** are smaller and closer to 0 , indicating a better fit for this class.

## 0.1 C.

Starting with initial weights/biases equal to a random number in [-0.5,. . . , 0.5], calculate the final (after convergence) weights/biases of the ADALINE using the LMS training algorithm with learning rate=0.001.

w[0] = [-0.28189291 0.35125034]
b[0] = 0.20605674458003675
We get these values using the function "np.random", so they might change every time we run the code.

```
present pattern p1 = [[0]
 [0]]
w[1] = [-0.28189291  0.35125034]
b[1] = 0.20805674458003676
a1[0] = 0.0
e1[0] = 1.0
------------------------------
present pattern p2 = [[0]
 [1]]
w[2] = [-0.28189291  0.35254784]
b[2] = 0.20935424390762503
a1[1] = 0.35125033620586676
e1[1] = 0.6487496637941332
------------------------------
present pattern p3 = [[1]
 [0]]
w[3] = [-0.27932913  0.35254784]
b[3] = 0.21191802973306992
a1[2] = -0.2818929127224462
e1[2] = 1.281892912722446
------------------------------
present pattern p4 = [[-1]
 [-1]]
w[4] = [-0.28147556  0.3504014 ]
b[4] = 0.21406446715034283
a1[3] = -0.07321870863645374
e1[3] = 1.0732187086364537
------------------------------
End for class A
```

Figure 15: present patterns for class A

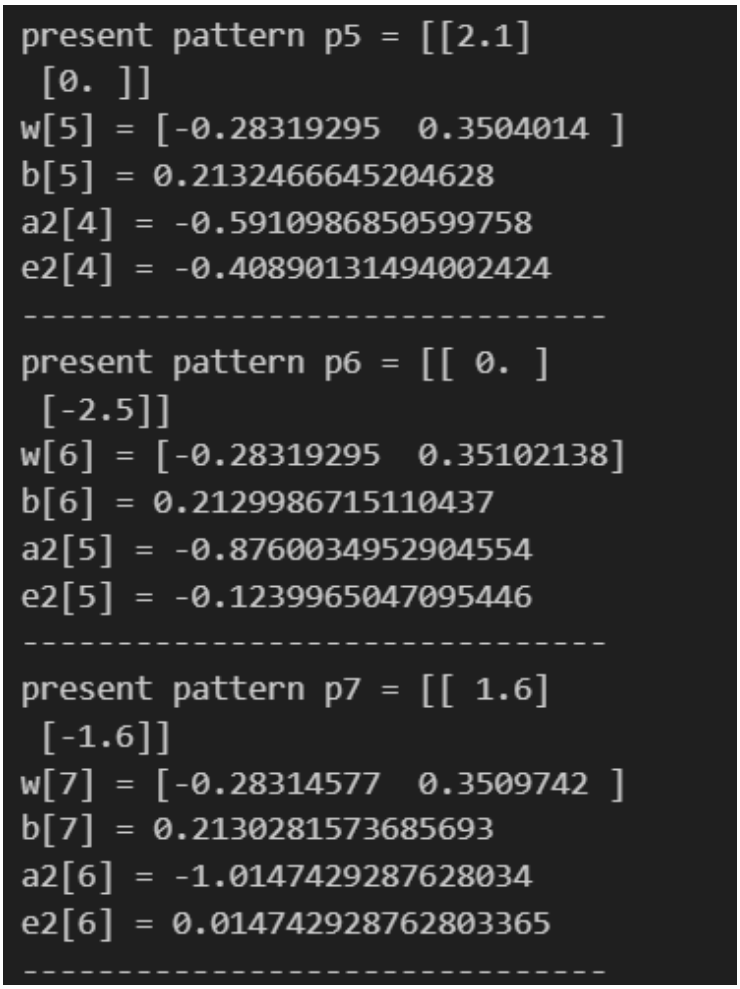



Figure 16: present patterns for class B

## Cocnlusion

To sum up, as long as the **error decreases in every pattern's presentation** and in the last presentation i have the smallest error, the **learning algorithm is successfully adjusting the weights and biases to minimize the discrepancy between the predicted and actual values**. The reduction in error suggests **the model is learning effectively and optimizing the parameters toward a solution that fits the training data**. This behavior confirms the efficiency of the implemented weight update rule and highlights the system's ability to approach a state of minimal error with successive iterations.

# Problem 10



**Problem-10**
Write code to implement the backpropagation algorithm for a 1-$S^1$-1 MLP network (logsigmoid-linear). Write the program using matrix operations, as we did in the class lecture. Choose the initial weights and biases to be random numbers uniformly distributed between -0.5 and 0.5, and train the network to approximate the function

$$g(p)=1+\sin[p(\pi/3)] \text{ for } -2 \le p \le 2.$$

Use $S^1= 2$, $S^1= 6$ and $S^1= 10$. Experiment with several different values for the learning rate $\alpha$, and use several different initial conditions. Discuss the convergence properties of the algorithm as the learning rate changes.

Figure 17: Data

## 1. $S^1 = 6$

**Learning Rate: a = 0.04**

**Updated W1:**

$$\begin{bmatrix} 0.43858844 \\ 0.2978349 \\ -0.26025319 \\ 0.26513273 \\ -0.32612564 \\ -0.34291197 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} 0.31984089 \\ -0.28642713 \\ 0.11209822 \\ -0.18559909 \\ -0.02309126 \\ -0.08285459 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.16851349 & -0.09747786 & -0.29813708 & 0.25464868 & 0.41024958 & -0.1547768 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} -0.2927099 \end{bmatrix}$$

**Error:**

2.5072748556913287

---

**Learning Rate: a = 0.08**

**Updated W1:**

$$\begin{bmatrix} 0.43998579 \\ 0.2878412 \\ -0.27990083 \\ 0.27270336 \\ -0.31022023 \\ -0.354236 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} 0.32123824 \\ -0.29642082 \\ 0.09245057 \\ -0.17802846 \\ -0.00718585 \\ -0.09417861 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.30499052 & 0.00438733 & -0.20329952 & 0.35816754 & 0.49166161 & -0.07443102 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} -0.09212791 \end{bmatrix}$$

**Error:**

2.5072748556913287

---

**Learning Rate: a = 0.1**

**Updated W1:**

$$\begin{bmatrix} 0.44068446 \\ 0.28284436 \\ -0.28972465 \\ 0.27648867 \\ -0.30226752 \\ -0.35989801 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} 0.32193691 \\ -0.30141767 \\ 0.08262675 \\ -0.17424315 \\ 0.00076685 \\ -0.09984062 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.37322904 & 0.05531993 & -0.15588075 & 0.40992698 & 0.53236762 & -0.03425812 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.00816308 \end{bmatrix}$$

**Error:**

2.5072748556913287

---

**Learning Rate: a = 0.3**

**Updated W1:**

$$\begin{bmatrix} 0.44767119 \\ 0.23287589 \\ -0.38796287 \\ 0.31434182 \\ -0.22274048 \\ -0.41651813 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} 0.32892364 \\ -0.35138614 \\ -0.01561146 \\ -0.13639 \\ 0.0802939 \\ -0.15646074 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 1.05561419 & 0.56464588 & 0.31830703 & 0.92752131 & 0.93942776 & 0.36747081 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 1.01107302 \end{bmatrix}$$

**Error:**

2.5072748556913287

**2.** $S^1 = 2$

**Learning Rate: 0.04**

**Updated W1:**

$$\begin{bmatrix} 0.20343306 \\ 0.19078972 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.41293605 \\ -0.16835339 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.15471752 & 0.04327787 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.48133687 \end{bmatrix}$$

**Error:**

1.4641816905021723

---

**Learning Rate: 0.08**

**Updated W1:**

$$\begin{bmatrix} 0.20639818 \\ 0.19032201 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.40997092 \\ -0.1688211 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.20700041 & 0.10252951 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.5984714 \end{bmatrix}$$

**Error:**

1.4641816905021723

---

**Learning Rate: 0.1**

**Updated W1:**

$$\begin{bmatrix} 0.20788074 \\ 0.19008816 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.40848836 \\ -0.16905495 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.23314185 & 0.13215534 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.65703867 \end{bmatrix}$$

**Error:**

1.4641816905021723

---

**Learning Rate: 0.3**

**Updated W1:**

$$\begin{bmatrix} 0.22270635 \\ 0.18774963 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.39366276 \\ -0.17139348 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.49455629 & 0.42841357 \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 1.24271134 \end{bmatrix}$$

**Error:**

1.4641816905021723

---

**3.** $S^1 = 10$

**Learning Rate: 0.04**

**Updated W1:**

$$\begin{bmatrix} 0.10126052 \\ 0.34270615 \\ 0.06668728 \\ -0.02658384 \\ -0.39947392 \\ -0.15019336 \\ 0.45826054 \\ -0.33333486 \\ 0.27486337 \\ -0.1705397 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.44238036 \\ -0.19845613 \\ -0.1008831 \\ 0.26656728 \\ -0.30634326 \\ -0.46884027 \\ -0.35904135 \\ -0.14017723 \\ 0.24221524 \\ -0.25589145 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} -0.03396361 & -0.04415968 & 0.04562346 & -0.16483557 & 0.16056717 & 0.29657034 \\ 0.02663073 & -0.30367004 & 0.41317169 & 0.34134616 & & \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.30625815 \end{bmatrix}$$

**Error:**

1.6516109810319073

---

**Learning Rate: 0.08**

**Updated W1:**

$$\begin{bmatrix} 0.09840014 \\ 0.33892171 \\ 0.06604816 \\ -0.03436065 \\ -0.39605834 \\ -0.14268947 \\ 0.45685039 \\ -0.34448658 \\ 0.28515106 \\ -0.16142711 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.44524074 \\ -0.20224057 \\ -0.10152223 \\ 0.25879047 \\ -0.30292769 \\ -0.46133638 \\ -0.3604515 \\ -0.15132895 \\ 0.25250293 \\ -0.24677885 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.02112455 & 0.02691007 & 0.11060065 & -0.0903758 & 0.20403941 & 0.34236552 \\ 0.09606284 & -0.2522622 & 0.49530792 & 0.39295993 & & \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.43838703 \end{bmatrix}$$

**Error:**

1.6516109810319073

---

**Learning Rate: 0.1**

**Updated W1:**

$$\begin{bmatrix} 0.09696995 \\ 0.3370295 \\ 0.0657286 \\ -0.03824905 \\ -0.39435056 \\ -0.13893752 \\ 0.45614531 \\ -0.35006245 \\ 0.2902949 \\ -0.15687081 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.44667093 \\ -0.20413278 \\ -0.10184179 \\ 0.25490207 \\ -0.3012199 \\ -0.45758443 \\ -0.36115658 \\ -0.15690482 \\ 0.25764677 \\ -0.24222255 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.04866863 & 0.06244494 & 0.14308925 & -0.05314592 & 0.22577553 & 0.3652631 \\ 0.13077889 & -0.22655828 & 0.53637603 & 0.41876682 & & \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 0.50445147 \end{bmatrix}$$

**Error:**

1.6516109810319073

---

**Learning Rate: 0.3**

**Updated W1:**

$$\begin{bmatrix} 0.08266806 \\ 0.31810732 \\ 0.06253298 \\ -0.07713309 \\ -0.37727269 \\ -0.10141803 \\ 0.44909456 \\ -0.40582106 \\ 0.34173333 \\ -0.11130781 \end{bmatrix}$$

**Updated b1:**

$$\begin{bmatrix} -0.46097282 \\ -0.22305496 \\ -0.10503741 \\ 0.21601803 \\ -0.28414203 \\ -0.42006494 \\ -0.36820733 \\ -0.21266343 \\ 0.3090852 \\ -0.19665955 \end{bmatrix}$$

**Updated W2:**

$$\begin{bmatrix} 0.32410947 & 0.41779366 & 0.46797524 & 0.31915288 & 0.44313673 & 0.59423896 \\ 0.47793944 & 0.0304809 & 0.94705718 & 0.67683567 & & \end{bmatrix}$$

**Updated b2:**

$$\begin{bmatrix} 1.16509586 \end{bmatrix}$$

**Error:**

1.6516109810319073

## Conclusion

- The updated weights ($W1$, $W2$) and biases ($b1$, $b2$) show that the network adjusts its parameters during training.
- Larger learning rates ($\alpha = 0.3$) result in more significant weight changes, whereas smaller learning rates ($\alpha = 0.04$) provide more stable updates.

- The error remains constant at approximately **1.464** or **2.507**, regardless of the learning rate, which suggests the network might not be fully converging to the desired function approximation.
- This could be due to parameter initialization, insufficient training iterations, or an overly simple network architecture.

---

## Convergence Properties

**1. Learning Rate ($\alpha$)**

- Smaller values of $\alpha$ (0.04) lead to slower updates and stable convergence but require more iterations to reach a minimum error.
- Larger $\alpha$ (0.3) results in faster updates but can lead to overshooting or instability, especially with steep gradients.

**2. Hidden Layer Size ($S^1$)**

- The network approximates the function better with larger hidden layers. For $S^1 = 6$ and $S^1 = 10$, the updates to weights and biases suggest more complex interactions, leading to better function approximation.

# Problem 11

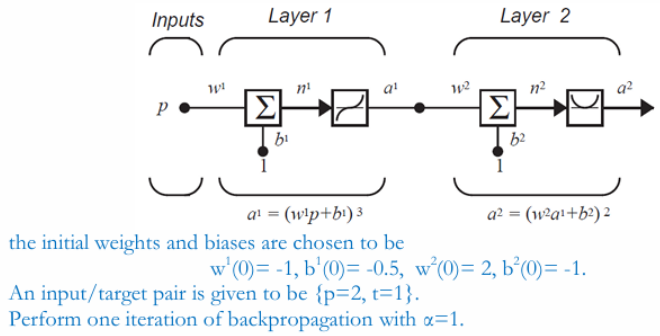



Figure 18: data

1. **Forward pass**

$a^0 = p \rightarrow a^0 = 2$
$a^1 = (w^1 * p + b^1)^3 \rightarrow a^1 = (w^1(0) * p + b^1(0))^3 \rightarrow a^1 = -15.625$

$a^2 = (w^2 * a^1 + b^2)^2 \rightarrow a^2 = (w^2(0) * a^1 + b^2(0))^2 \rightarrow a^2 = 1040.06$

**error:** $e = t - a^2 = 1 - 1040.06 \rightarrow e = -1039.06$

2. **Backward pass**

$f^2(x) = x^2$
$f'^2(x) = 2 * x = 2\sqrt{f}$, where $f = a^2 = 1040.06$

- $s^2 = -2 * f'^M(n^M) * (t - a^2) = -2 * f'^2(n^2) * (t - a^2) \rightarrow$
  $s^2 = -2 * (2\sqrt{1040.06})(-1039.06) \rightarrow s^2 = 134,038$
- $s^1 = f'^m(n^m) * w^{m+1} * s^{m+1}$
  $f^1(x) = x^3$
  $f'^1(x) = 3 * x^2 = 3f^{\frac{2}{3}}$, where f=-15.625

  So, $s^1 = f'^1(n^1) * w^2(0) * s^2 \rightarrow s^1 = (3 * (-15.625)^{\frac{2}{3}}) * 2 * 134,038 \rightarrow$
  $s^1 = 5,026,425$

3. **weights and bias**

- $w^2(1) = w^2(0) - a * s^2 * a^1 = 2 - 1 * 134,038 * (-15.625) \rightarrow$
  $w^2(1) = 2,094,345.75$
- $b^2(1) = b^2(0) - a * s^2 = (-1) - 1 * 134,038 \rightarrow b^2(1) = -134,039$
- $w^1(1) = w^1(0) - a * s^1 * a^1 = -1 - 1 * 5,026,425 * (-15.625) \rightarrow$
  $w^1(1) = 78,537,889.63$
- $b^1(1) = b^1(0) - a * s^1 = -0.5 - 1 * 5,026,425 \rightarrow b^1(1) = -5,026,425.5$